CH3

API Java et programmation avancée

- Gestion des exceptions
- Collections : List, Set, Map
- Dates, fichiers et introduction aux Streams
- String et manipulation de chaînes
- Généricité et itération

Gestion des exceptions : try / catch / finally

- ✓ Une exception est une interruption de l'exécution d'un programme suite à une erreur.
- ✓ Les exceptions représentent le mécanisme de gestion des erreurs intégré au langage Java.
- ✓ Pour gérer l'exception, on peut utiliser le bloc **try** et **catch** entourant le code qui peut provoquer l'exception.
- ✓ Par exemple : Une division par zéro provoque une exception de type *ArithmeticException*.

```
try
{
 //statement1;
 //statement2;
 //statement3;
 //...
}
catch(exception-type)
{
 //code that handles exception.
}
```

Enclosing statements in try block *may* throw an exception.

catch block catches and handles a *type* of an exception.

```
public class Exemple {
public static void main(String[] args) {
int res = 10 / 0 ;
System.out.println("Valeur de res est : " + res);
}
}
```

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at Exemple.main(Exemple.java:4)
```

```
public class Exemple {
   public static void main(String[] args) {
      try {
         int res = 10/0;
      } catch (ArithmeticException e) {
         e.printStackTrace();
         System.out.println("NumberFormatException est traité");
      }
      System.out.println("Continuez votre code ici");
   }
}
```

```
NumberFormatException est traité
Continuez votre code ici
```

Gestion des exceptions : try / catch / finally

```
public double diviser(int a, int b) {
    try {
        return a / b;
    } catch (ArithmeticException e) {
        System.out.println("Erreur : " + e.getMessage());
        return 0;
    } finally {
        System.out.println("Opération terminée");
    }
}

// Exception personnalisée
public class MoyenneInvalideException extends Exception {
    public MoyenneInvalideException(String message) {
        super(message);
    }
}
```

Mécanisme

- try : bloc de code susceptible de lever une exception
- catch : intercepte et traite un type d'exception précis
- finally : s'exécute toujours (erreur ou non) — libération de ressources
- throw lève une exception ; throws déclare qu'une méthode peut en lever une
- Créer sa propre exception : extends Exception (vérifiée) ou RuntimeException

Hiérarchie des exceptions Java

Catégorie	Exemple	Doit être déclarée (throws) ?
Erreurs système (Error)	OutOfMemoryError, StackOverflowError	Non — non récupérable
Exceptions vérifiées (checked)	IOException, SQLException	Oui — obligatoire (try/catch ou throws)
Exceptions non vérifiées (unchecked)	NullPointerException, ArithmeticException	Non — RuntimeException
Exceptions personnalisées	MoyenneInvalideException	Selon le parent choisi (Exception ou RuntimeException)

Toutes les exceptions héritent de Throwable. Bonne pratique : ne jamais attraper Exception ou Throwable de façon générique sans raison.

String et manipulation de chaînes

```
String nom = "Amine";
String complet = nom + " Benali";      // concaténation
System.out.println(complet.length());  // longueur
System.out.println(complet.toUpperCase());
System.out.println(complet.substring(0, 5));
System.out.println(complet.contains("Ben"));
System.out.println(complet.equals("Amine Benali")); // pas
==

StringBuilder sb = new StringBuilder();
for (int i = 0; i < notes.length; i++) {
    sb.append(notes[i]).append(", ");
}
String resultat = sb.toString();
```

Points essentiels

- String est immuable : toute "modification" crée une nouvelle chaîne
- Comparer le contenu avec equals(), jamais == (qui compare les références)
- Méthodes utiles : length(), substring(), split(), trim(), replace(), indexOf()
- StringBuilder : concaténations multiples dans une boucle, plus performant que +
- String.format() ou printf() pour un affichage formaté

Le Java Collections Framework

1

List<E> (ArrayList, LinkedList)

Collection ordonnée, avec doublons possibles, accès par index. ArrayList : implémentation la plus courante.

2

Set<E> (HashSet, TreeSet)

Collection sans doublons. HashSet : ordre non garanti et rapide. TreeSet : éléments triés automatiquement.

3

Map<K,V> (HashMap, TreeMap)

Associe une clé unique à une valeur. Très utilisé pour indexer des objets par identifiant.

4

Pourquoi pas des tableaux ?

Taille dynamique, méthodes riches (add, remove, contains, sort…), et compatibilité avec les génériques.

5

Interfaces vs implémentations

On programme avec l'interface (List<Etudiant> liste = new ArrayList<>();) pour rester flexible.

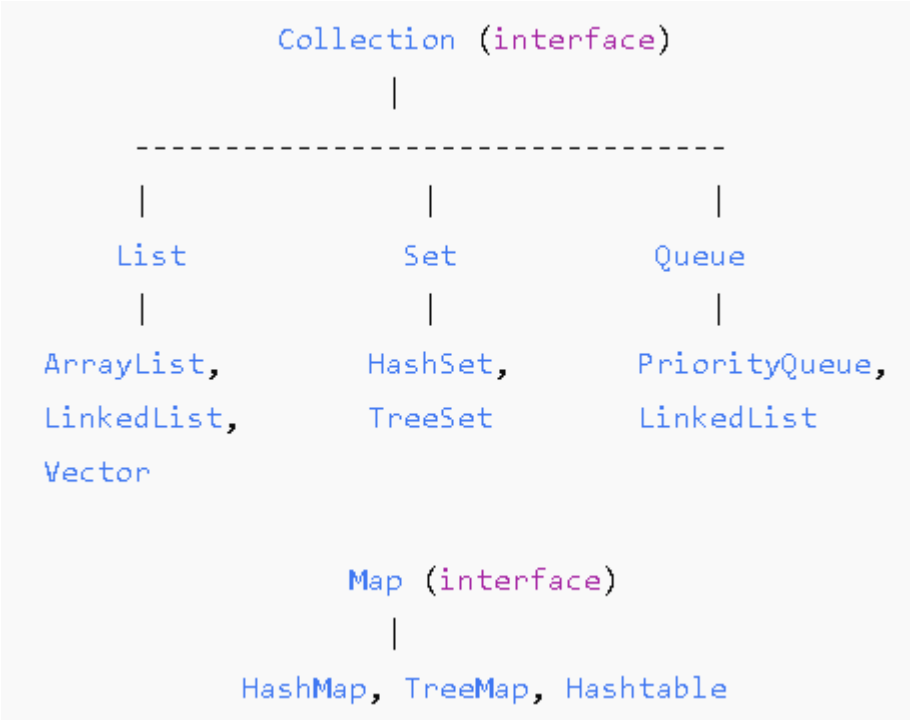
6

Tri et recherche

Collections.sort(liste), Collections.max/min, ou implémenter Comparable<T> sur ses propres classes.

Collection

- ✓ Une **Collection** en Java est un **ensemble d'objets regroupés dans une même structure de données**.
- ✓ La structure de collections fournit un ensemble d'interfaces et de classes permettant : le stockage dynamique d'objets, le tri, la recherche, la suppression, et la parcours à l'aide d'itérateurs.
- ✓ Les collections remplacent avantageusement ***les tableaux (arrays)*** qui ont une taille fixe



Collection : List

- ✓ Une List stocke les éléments dans un ordre précis, et peut contenir des doublons.

- **Implémentations :**
- ✓ **ArrayList** → rapide en lecture, lente en insertion au milieu.
- ✓ **LinkedList** → rapide en insertion/suppression, lente en accès aléatoire.
- ✓ **Vector** → similaire à ArrayList mais synchronisé (ancien).

```java
import java.util.*;
public class ExempleList {
    public static void main(String[] args) {
        List<String> noms = new ArrayList<>();
        noms.add("Alice");
        noms.add("Bob");
        noms.add("Charlie");
        noms.add("Alice"); // doublon autorisé
        System.out.println("Liste : " + noms);
        System.out.println("Élément à l'indice 1 : " + noms.get(1));
    }
}
```

Collection : Set

- ✓ Un **Set** ne contient aucun doublon et n'a pas d'ordre fixe (sauf **TreeSet**).

Implémentations :

- ✓ **HashSet** → rapide, ordre non garanti.
- ✓ **LinkedHashSet** → conserve l'ordre d'insertion.
- ✓ **TreeSet** → trie automatiquement les éléments.

```java
import java.util.*;
public class ExempleSet {
    public static void main(String[] args) {
        Set<String> fruits = new HashSet<>();
        fruits.add("Pomme");
        fruits.add("Orange");
        fruits.add("Banane");
        fruits.add("Orange"); // ignoré
        System.out.println("Set : " + fruits);
    }
}
```

Collection : Map

- ✓ Une **Map** stocke des paires (***clé***, ***valeur***).
- ✓ Chaque **clé** est unique, mais plusieurs clés peuvent avoir la même **valeur**.

Implémentations :

- ✓ **HashMap** → rapide, ordre non garanti.
- ✓ **LinkedHashMap** → conserve l'ordre d'insertion.
- ✓ **TreeMap** → trie les clés automatiquement

```java
import java.util.*;
public class ExempleMap {
    public static void main(String[] args) {
        Map<Integer, String> etudiants = new HashMap<>();
        etudiants.put(1, "Alice");
        etudiants.put(2, "Bob");
        etudiants.put(3, "Charlie");
        System.out.println("Map des étudiants : " + etudiants);
        System.out.println("Nom de l'étudiant 2 : " + etudiants.get(2));
    }
}
```

Collection : Map

- ✓ Une **Map** stocke des paires (***clé***, ***valeur***).
- ✓ Chaque **clé** est unique, mais plusieurs clés peuvent avoir la même **valeur**.

Implémentations :

- ✓ **HashMap** → rapide, ordre non garanti.
- ✓ **LinkedHashMap** → conserve l'ordre d'insertion.
- ✓ **TreeMap** → trie les clés automatiquement

```java
import java.util.*;
public class ExempleMap {
    public static void main(String[] args) {
        Map<Integer, String> etudiants = new HashMap<>();
        etudiants.put(1, "Alice");
        etudiants.put(2, "Bob");
        etudiants.put(3, "Charlie");
        System.out.println("Map des étudiants : " + etudiants);
        System.out.println("Nom de l'étudiant 2 : " + etudiants.get(2));
    }
}
```

Collection : Parcourir et Manipuler

Avec `for-each`

```java
for (String nom : noms) {
    System.out.println(nom);
}
```

Avec un `Iterator`

```java
Iterator<String> it = noms.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

Méthode	Description
add(E e)	Ajoute un élément
remove(Object o)	Supprime un élément
clear()	Vide la collection
size()	Renvoie le nombre d'éléments
contains(Object o)	Vérifie la présence d'un élément
isEmpty()	Vérifie si la collection est vide

Collection : Parcourir et Manipuler

Avantages des Collections :

- ✓ Manipulation flexible de données
- ✓ Structures de données réutilisables
- ✓ Tri et recherche facilités (Collections.sort(), contains())
- ✓ Compatible avec les flux (Streams) introduits en Java 8

Type	Caractéristique principale	Doublons	Ordre
List	Liste ordonnée d'éléments	Oui	Oui
Set	Ensemble unique d'éléments	Non	Non (sauf TreeSet)
Map	Paires clé/valeur	(clés uniques)	Variable selon implémentation

List, Map et généricité

```
List<Etudiant> etudiants = new ArrayList<>();
etudiants.add(new Etudiant("E001", "Amine"));
etudiants.add(new Etudiant("E002", "Sara"));

for (Etudiant e : etudiants) {          // for-each
    System.out.println(e.getNom());
}

Map<String, Etudiant> parMatricule = new HashMap<>();
parMatricule.put("E001", etudiants.get(0));
Etudiant e = parMatricule.get("E001");

// Généricité : la classe travaille avec n'importe quel
type
public class Boite<T> {
    private T contenu;
    public void set(T v) { this.contenu = v; }
    public T get() { return contenu; }
}
```

À retenir

- <Etudiant> : type générique — garantit la sécurité de type à la compilation
- for-each : parcours simplifié, préférable à une boucle indexée classique
- Map.put(clé, valeur) / Map.get(clé) : accès direct en O(1) avec HashMap
- Une classe générique <T> peut être réutilisée avec différents types d'objets
- Iterator<E> permet un parcours explicite avec suppression sécurisée en cours de boucle

Choisir la bonne collection

Besoin	Collection recommandée	Pourquoi
Liste ordonnée, doublons autorisés	ArrayList<E>	Accès rapide par index, ajout en fin efficace
Pas de doublons, ordre indifférent	HashSet<E>	Recherche/ajout très rapides (table de hachage)
Pas de doublons, éléments triés	TreeSet<E>	Tri automatique selon l'ordre naturel ou un Comparator
Association clé → valeur	HashMap<K,V>	Recherche par clé en temps quasi constant
Association triée par clé	TreeMap<K,V>	Parcours des clés dans l'ordre croissant

Manipulation de fichiers

- ✓ La création et l'écriture de fichiers en Java est une tâche fondamentale pour de nombreuses applications.
- ✓ Java fournit plusieurs classes et méthodes pour gérer efficacement les opérations sur les fichiers.
- ✓ La **classe File Java** permet de gérer les fichiers, les noms de chemins de fichiers et de répertoires de manière abstraite. Cette classe est utilisée pour la création de fichiers et de répertoires, la recherche de fichiers, la suppression de fichiers, etc.



Manipulation de fichiers : Création

- ✓ Pour créer un fichier en Java, c'est à l'aide de la classe ***File*** et la méthode ***createNewFile()***.
- ✓ Cette méthode crée un fichier vide, s'il n'existe pas à l'emplacement spécifié et renvoie **true**. Si le fichier est déjà présent, cette méthode renvoie **false**.
- ✓ ***Exemple :*** Créer un simple fichier texte
  1. Importer les bibliothèques **File** et **IOException** qui se trouvent dans le **package java.io**
  2. Instanciation sur la **classe File** pour créer un nouveau **objet myFile** du type **file**
  3. Crée le fichier avec la méthode **: createNewFile() (myFile.createNewFile())**
- ✓ **IOException** : Si une erreur d'entrée / sortie survient lors de la création du fichier.

```java
import java.io.File;
import java.io.IOException;
public class NewFile
{
   public static void main( String[] args )
   {
      try {
            File myFile = new File("F:/FS/Mes Cours/JAVA/Exp/9 File/newFile.txt");
            /* Si le fichier est créé, alors la méthode createNewFile() renverrait
             true et  si le fichier est déjà présent il retournerait faux */
               boolean varF = myFile.createNewFile();
            if (varF){
                System.out.println("Fichier crée avec succés");
            }
            else{
                System.out.println("Le fichier existe déjà");
            }
           } catch (IOException e) {
              System.out.println("Exception Occurred:");
              e.printStackTrace();
          }
   }
}
```

Manipulation de fichiers : Écriture

- ✓ La classe ***FileWriter*** permet d'ouvrir un flux d'écriture sur un fichier via son constructeur :
- **FileWriter**(String ***nom_du_fichier***, boolean ***append***)

- ✓ Si la valeur du boolean ***append*** est mis à **false**, le contenu du fichier sera écrasé par le nouveau contenu
- ✓ Si la valeur du boolean ***append*** est mis à **true**, le nouveau contenu sera ajouté à l'ancien

```java
import java.io.FileWriter;
import java.io.IOException;
public class WriteFile
{
   public static void main( String[] args )
   {
    try
    {
        String filename= "F:/FS/Mes Cours/JAVA/Exp/9 File/newFile.txt";
        FileWriter fw = new FileWriter(filename,true);
        //la valeur true entraine l'ajout du nouveau contenu à l'ancien
        fw.write("Voici le contenu qui va être ajouté au fichier ");
        //appends the string to the file
        fw.close();
    }
    catch(IOException ioe)
    {
        System.err.println("IOException: " + ioe.getMessage());
    }
   }
}
```

Manipulation de fichiers : Lecture

- ✓ **FileReader**: Utilisé pour lire des fichiers texte de manière simple et efficace.
- ✓ **BufferedReader**: S'enroule autour de ***FileReader*** pour lire du texte à partir d'un flux d'entrée de caractères, en mettant les caractères en mémoire tampon pour une lecture efficace des caractères, des tableaux et des lignes.

```
PS F:\FS\Mes Cours\JAVA\Exp\9 File> javac ReadFile.java
PS F:\FS\Mes Cours\JAVA\Exp\9 File> java ReadFile
Voici le contenu qui va être ajouté au fichier
```

```java
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
public class ReadFile
{
   public static void main( String[] args )
   {
    String filename= "F:/FS/Mes Cours/JAVA/Exp/9 File/newFile.txt";
    try (FileReader fileReader = new FileReader(filename);
        BufferedReader bufferedReader = new BufferedReader(fileReader)) {
        String line;
        while ((line = bufferedReader.readLine()) != null) {
          System.out.println(line);
        }
       }
    catch (IOException e) {
     System.out.println("Error reading file: " + e.getMessage());
    }
   }
}
```

Manipulation de fichiers : Supprimer

- ✓ Pour supprimer un fichier existant en Java, il suffit d'instancier la classe **FileWriter** et pointer vers le fichier, et appliquer ensuite la méthode **delete()** à l'objet :

```java
import java.io.File;
public class DeleteFile
{
   public static void main( String[] args )
   {
    String filename= "F:/FS/Mes Cours/JAVA/Exp/9 File/newFile1.txt";
    File myFile = new File(filename);
    myFile.delete();
   }
}
```

java.time et manipulation de fichiers

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

LocalDate aujourdHui = LocalDate.now();
LocalDate inscription = LocalDate.of(2026, 9, 15);
DateTimeFormatter fmt =
DateTimeFormatter.ofPattern("dd/MM/yyyy");
System.out.println(inscription.format(fmt));

// Écrire dans un fichier
try (BufferedWriter bw = new BufferedWriter(new
FileWriter("etudiants.txt"))) {
    bw.write("E001;Amine;14.5");
    bw.newLine();
} catch (IOException e) {
    System.out.println("Erreur d'écriture : " +
e.getMessage());
}
```

À retenir

- java.time (LocalDate, LocalDateTime) remplace l'ancienne classe Date, jugée peu fiable
- DateTimeFormatter pour l'affichage et le parsing de dates dans un format donné
- try-with-resources : ferme automatiquement le flux (BufferedWriter, Scanner de fichier…)
- IOException est une exception vérifiée : gestion obligatoire
- Utile pour la sauvegarde simple avant d'introduire JDBC au chapitre suivant

TP 10 : Gestion des étudiants (Fichier)

Créer une application en Java pour la gestion des étudiants en utilisant un fichier texte «**Etudiant.txt**»:

1. Créer le fichier ***Etudiant.txt***
2. Saisir les informations d'un étudiant (Numéro, Nom, Prénom, Email, Note)
3. Afficher la liste des étudiants enregistrés
4. Supprimer le fichier ***Etudiant.txt***

Avec l'utilisation d'un switch pour naviguer entre les choix.

Introduction aux Streams (Java 8+)

```java
List<Etudiant> etudiants = List.of(
    new Etudiant("Amine", 14.5),
    new Etudiant("Sara", 9.0),
    new Etudiant("Youssef", 16.0)
);

List<String> admis = etudiants.stream()
    .filter(e -> e.getMoyenne() >= 10)
    .map(Etudiant::getNom)
    .sorted()
    .collect(Collectors.toList());

double moyenneGenerale = etudiants.stream()
    .mapToDouble(Etudiant::getMoyenne)
    .average()
    .orElse(0);
```

Principe

- Un Stream traite une collection comme un flux d'opérations chaînées
- filter() : sélectionne des éléments selon une condition (prédicat)
- map() : transforme chaque élément (ex : Etudiant → nom)
- collect() : reconstruit une collection à partir du flux
- Notation vue ici comme introduction ; approfondie en travaux pratiques ultérieurs

SYNTHÈSE

Synthèse CH3

- ✓ try/catch/finally structure la gestion des erreurs prévisibles
- ✓ List, Set, Map : trois familles de collections aux usages différents
- ✓ java.time gère les dates ; try-with-resources sécurise les flux de fichiers
- ✓ Ces briques (collections, exceptions, fichiers) préparent directement JDBC (CH5)
- ✓ String est immuable ; StringBuilder pour les concaténations répétées
- ✓ La généricité <T> sécurise le typage sans dupliquer le code
- ✓ Les Streams permettent de filtrer/transformer une collection de façon déclarative

TP5 (S5) : gestion d'une liste d'étudiants (ArrayList, tri, filtrage, sauvegarde fichier) + débogage assisté par IA